

집합(Set)과 맵(Map) - 중복 없이, 키로 바로 찾기

왜 필요할까?

배열은 "몇 번째"로 찾는다. 집합과 맵은 "값이 있는가?" "이름으로 찾기"를 빠르게 한다. KOI에서 "중복 제거", "등장 횟수 세기", "이름→점수"는 전부 이 둘로 푼다.

비유 - 집합(Set): 출석부. 이름이 한 번만 적힌다. "영희가 있는가?"만 본다. - 맵(Map): 성적표. "영희 → 90 점"처럼 키와 값이 짝을 이룬다.

```
flowchart LR
    subgraph Set
        S1[1, 2, 3] --> S2[2를 또 넣어도 1,2,3 그대로\ 중복 자동 제거]
    end
    subgraph Map
        M1[사과→3개] --> M2[바나나→5개]
        M2 --> M3[사과를 찾으면 3이 바로 나옴]
    end
```

1. 집합(Set) - 중복 없이, 정렬된 채로 보관

C++ set은 정렬 + 중복 제거 + $O(\log n)$ 탐색을 한다.

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    set<int> s;
    s.insert(3);
    s.insert(1);
    s.insert(2);
    s.insert(3); // 중복이라 무시됨
```

```

for (int x : s) cout << x << ' '; // 1 2 3 (자동 정렬)
cout << '\n';

if (s.find(2) != s.end()) cout << "2 있음\n";
cout << "크기: " << s.size() << '\n'; // 3
s.erase(2);
cout << "2 삭제 후 크기: " << s.size() << '\n'; // 2
}

```

flowchart TD

```

A[insert 3] --> B[트리: 3]
B --> C[insert 1 → 1이 3 왼쪽]
C --> D[insert 2 → 2가 1 오른쪽, 3 왼쪽]
D --> E[항상 정렬 유지]

```

연산	시간	설명
insert(x)	$O(\log n)$	정렬 위치에 넣음
find(x)	$O(\log n)$	있으면 위치, 없으면 end
erase(x)	$O(\log n)$	삭제
size()	$O(1)$	개수

더 빠른 집합이 필요하면 `unordered_set`. 평균 $O(1)$ 이지만 정렬은 안 된다.

2. 맵(Map) – 키로 값 찾기

`map<key, value>`는 키를 정렬해 보관한다. 키로 값을 바로 찾는다.

```

#include <bits/stdc++.h>
using namespace std;

```

```
int main() {
    map<string, int> score;
    score["영희"] = 90;
    score["철수"] = 80;
    score["영희"] = 95; // 같은 키에 덮어쓰기

    if (score.find("영희") != score.end())
        cout << "영희: " << score["영희"] << '\n'; // 95

    for (auto &p : score)
        cout << p.first << " " << p.second << '\n';
    // 영희 95
    // 철수 80 (키 순 정렬)
}
```

flowchart TD

K1[키: 영희] --> V1[값: 90]

K2[키: 철수] --> V2[값: 80]

K1 -. -> |find 영희| V1

연산	시간	설명
<code>m[key] = val</code>	$O(\log n)$	넣거나 덮어쓰기
<code>find(key)</code>	$O(\log n)$	키가 있는가
<code>erase(key)</code>	$O(\log n)$	삭제

더 빠른 맵이 필요하다면 `unordered_map`. 평균 $O(1)$, 정렬 안 됨.

3. 언제 무엇을 쓰나

flowchart TD

Q[무엇이 필요한가?] --> A{중복을 빼야 하나?}

A --> |예| S[set / unordered_set]

```

A --> |아니오| B{키로 값을 찾아야 하나?}
B --> |예| M[map / unordered_map]
B --> |아니오| V[vector]

S --> S1{정렬이 필요한가?}
S1 --> |예| S2[set 0 log n]
S1 --> |아니오| S3[unordered_set 0 1]

M --> M1{정렬이 필요한가?}
M1 --> |예| M2[map 0 log n]
M1 --> |아니오| M3[unordered_map 0 1]

```

KOI 선택 기준

상황	추천
중복 제거 + 순서대로 출력	set
등장 횟수 세기, 빈도표	map 또는 unordered_map
빠른 존재 확인만, 순서 필요 없음	unordered_set
키로 빠르게 찾기, 순서 필요 없음	unordered_map

4. 예제: 등장 횟수 세기

"수 n개가 주어질 때 각 수가 몇 번 나오는가?"

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<int> a = {1, 2, 2, 3, 3, 3};
    map<int, int> cnt;
    for (int x : a) cnt[x]++; // 없으면 0에서 1 증가

    for (auto &p : cnt)
        cout << p.first << "은 " << p.second << "번\n";
    // 1은 1번, 2는 2번, 3은 3번
}

```

```
// 순서 필요 없고 더 빠르게: unordered_map
unordered_map<int,int> cnt2;
for (int x : a) cnt2[x]++;
```

5. 직접 손으로 풀어 보기

문제 1. `set<int> s = {5,1,3}`에 `insert(3)` `insert(4)`를 하면 `s`는?

▶ 풀이

문제 2. `map<string,int> m; m["a"]=1; m["b"]=2; m["a"]=3;`에서 `m["a"]`는?

▶ 풀이

6. KOI에서는 이렇게 나온다

- "서로 다른 수의 개수" → `set`에 넣고 `size()`만 보면 끝
- "각 숫자의 빈도", "가장 많이 나온 수" → `map`으로 세기
- "이름과 점수가 짝으로 주어진다" → `map<string,int>`가 정답 구조
- `n`이 20만 이상이고 정렬이 필요 없으면 `unordered_map`으로 바꿔 $O(\log n)$ 을 $O(1)$ 로 줄인다. 시간초과가 여기서 걸린다.